

Construção e Análise de Grafos de Colaboração a partir da Mineração de Repositórios GitHub

Vitor Fernandes de Souza¹, João Pedro Queiroz Rocha¹, Maria Eduarda¹

¹ Curso de Engenharia de Software – Instituto de Ciências Exatas e Informática
Pontifícia Universidade Católica de Minas Gerais (PUC Minas)
Belo Horizonte – MG – Brasil

vitor.souza@sga.pucminas.br, email2@pucminas.br, email3@pucminas.br

Abstract. *This meta-paper describes the style to be used in articles and short papers for SBC conferences. For papers in English, you should add just an abstract while for the papers in Portuguese, we also ask for an abstract in Portuguese (“resumo”). In both cases, abstracts should not have more than 10 lines and must be in the first page of the paper.*

Resumo. *Este meta-artigo descreve o estilo a ser usado na confecção de artigos e resumos de artigos para publicação nos anais das conferências organizadas pela SBC. É solicitada a escrita de resumo e abstract apenas para os artigos escritos em português. Artigos em inglês deverão apresentar apenas abstract. Nos dois casos, o autor deve tomar cuidado para que o resumo (e o abstract) não ultrapassem 10 linhas cada, sendo que ambos devem estar na primeira página do artigo.*

1. Introdução

Este trabalho prático da disciplina de Teoria dos Grafos tem como objetivo integrar os fundamentos teóricos estudados em sala com a aplicação prática desses conceitos por meio do desenvolvimento de uma biblioteca de grafos e da análise de dados reais extraídos da API (*Application Programming Interface*) do GitHub. A proposta central consiste em compreender como estruturas formais de grafos podem ser utilizadas para modelar e interpretar redes de colaboração presentes em projetos de software de código aberto.

Ao longo do projeto, foram implementadas estruturas de grafos dirigidos e ponderados utilizando listas e matrizes de adjacência, seguindo princípios de abstração, herança e encapsulamento. Essas estruturas servem como base para a construção de múltiplos grafos derivados das interações entre usuários em um repositório GitHub, incluindo comentários, revisões, fechamento de issues e merges de pull requests. Cada tipo de interação é convertido em uma aresta direcionada, com pesos que refletem a intensidade e relevância da relação entre os colaboradores.

Este relatório documenta todas as etapas do trabalho, desde a concepção e implementação da biblioteca de grafos até a extração, modelagem, análise e visualização dos dados coletados. Além da construção dos grafos individuais e do grafo integrado, são aplicadas métricas clássicas de análise de redes, como centralidades, detecção de comunidades e identificação de nós que atuam como pontes estruturais. Por fim, os resultados são visualizados por meio de arquivos exportados para o software Gephi, permitindo uma exploração mais rica da estrutura colaborativa observada.

A execução deste trabalho reforça a importância da Teoria dos Grafos na análise de sistemas complexos e evidencia sua aplicabilidade direta em cenários reais de engenharia de software, especialmente no estudo de dinâmicas colaborativas em ambientes distribuídos.

2. Metodologia

A metodologia adotada neste trabalho organiza-se em três etapas complementares que integram fundamentos teóricos, desenvolvimento computacional e análise empírica. A primeira etapa consiste na implementação de uma biblioteca própria para manipulação de grafos, garantindo a compreensão das estruturas fundamentais necessárias para as demais etapas. A biblioteca foi construída em conformidade com a especificação estabelecida no enunciado do trabalho prático, contemplando operações de criação, remoção e verificação de arestas, cálculo de graus, manipulação de pesos e validações estruturais que asseguram a integridade dos grafos. A implementação segue uma organização baseada em herança, na qual uma classe abstrata define a *Application Programming Interface* (API) e classes concretas empregam listas e matrizes de adjacência para representar a conectividade entre vértices. Essa etapa estabelece a infraestrutura técnica necessária para a representação e exploração das relações observadas nos dados coletados.

A segunda etapa dedica-se à extração de dados reais do repositório vercel/next.js por meio da API GraphQL do GitHub. A escolha desse repositório atende ao requisito de possuir elevado número de colaboradores e interações, o que possibilita a construção de um grafo expressivo para análise. A API fornece informações referentes a *issues*, *pull requests*, comentários e revisões, permitindo mapear diferentes formas de interação entre usuários. Os dados minerados foram organizados em estruturas tabulares e passaram por procedimentos de limpeza, padronização temporal, eliminação de duplicidades e normalização de identificadores, garantindo consistência na formação das arestas e reduzindo possíveis distorções decorrentes de ruídos nos dados.

A terceira etapa envolve a construção do grafo que representa a rede de colaboração entre usuários do repositório. Cada usuário é modelado como um vértice, enquanto cada interação constitui uma aresta direcionada. Os pesos atribuídos seguem os critérios definidos no enunciado: comentários recebem peso 2; ações de abertura ou fechamento de *issues* por outro usuário recebem peso 3; revisões e aprovações de *pull requests* recebem peso 4; e operações de *merge* recebem peso 5. Essa atribuição permite distinguir a intensidade das relações e analisar a estrutura colaborativa com maior precisão. Antes da criação do grafo integrado, os dados passaram por nova verificação para assegurar que o grafo permanecesse simples, sem laços e sem múltiplas arestas entre o mesmo par de usuários.

Após a construção do grafo, realizou-se a exportação para o software Gephi, utilizado para visualização de redes complexas. A visualização permitiu observar padrões estruturais, identificar agrupamentos naturais e examinar a distribuição de conexões entre usuários. Além disso, foram aplicadas métricas clássicas da teoria dos grafos, incluindo análise de graus ponderados, identificação de usuários que contribuem para maior fragmentação da rede, detecção de comunidades e mensuração de vizinhos diretos e indiretos. As funções responsáveis por essas análises foram implementadas a partir da biblioteca desenvolvida na primeira etapa, assegurando continuidade metodológica entre

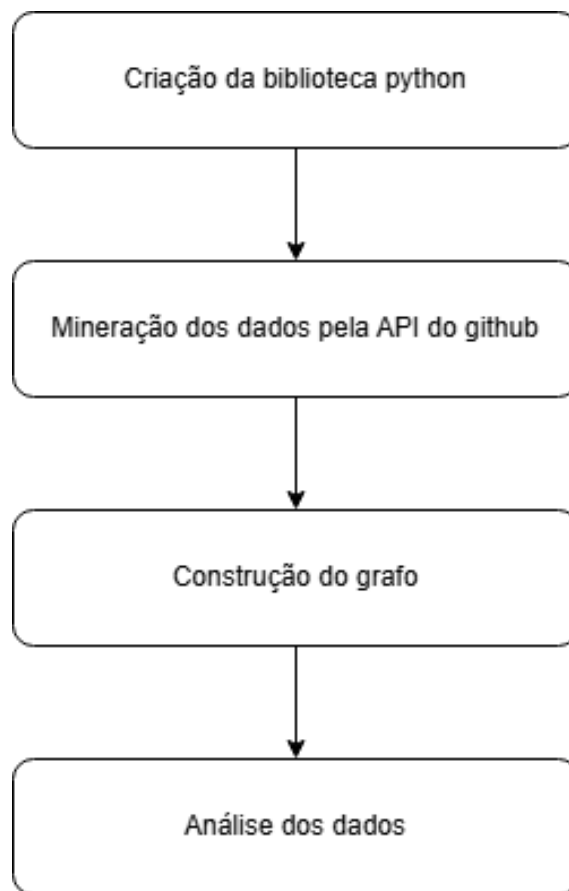


Figura 1. Metodologia

as fases de desenvolvimento e exploração dos dados.

A integração dessas três etapas resulta em uma abordagem que combina a implementação de estruturas fundamentais, a mineração sistemática de dados e a análise baseada em conceitos formais da teoria dos grafos. Essa metodologia permite interpretar o comportamento colaborativo no repositório estudado e demonstra a aplicabilidade prática dos conceitos da disciplina em cenários reais.

3. Resultados

3.1. Biblioteca de grafos

Nesta etapa foi desenvolvida uma biblioteca de grafos para executar as operações fundamentais e os algoritmos clássicos de análise, contando com uma interface visual integrada, dando mais agilidade na configuração. No desenvolvimento da biblioteca a linguagem escolhida foi o python.

As principais funcionalidades implementadas foram:

- Criação de vértices e arestas
- Atribuição de pesos e rótulos para vértices e arestas
- Geração do grafo final

3.2. Modelo de dados

O processo de mineração realizou a extração de 4 categorias centrais de dados conforme a Tabela 1. As categorias foram mineradas e armazenadas de modo a preservar as relações hierárquicas e cronológicas, garantindo maior acurácia na análise da rede.

Tabela 1. Modelo de dados minerados

Categoria	Campos
Interações	Número, autor, estado, comentarios, estado
Comments	Autor, body
Issue Closures	Estado
Reviews	Autor, Estado,
Merges	Mergeado, autor

3.3. Dados minerados

Os dados coletados durante o processo de mineração resultaram em um dataset extenso conforme apresentado na Tabela 2 e 3

Tabela 2. Dados da Mineração do GitHub

Métrica	Valor
Total de Usuários	2.579
Total de Interações	26.981
Pares Únicos de Usuários	4.620
Peso Total das Interações	77.978.0

Tabela 3. Tipos de Interações Mineradas

Tipo de Interação	Quantidade	Percentual
Comments	15.128	56.1%
Reviews	10.227	37.9%
Merges	968	3.6%
Close	658	2.4%
Total	26.981	100,0%

A coleta de dados abrangeu as *issues* e *pull request* mais recentes no repositório, coletando 77.978 interações em aproximadamente 5.000 requests à api. Os dados foram armazenados em um JSON estruturado, permitindo o consumo posterior das informações e reaproveitamento dos dados para a análise, garantindo a conformidade da estrutura de dados entre as operações.

3.4. Construção do grafo

3.4.1. Representação dos grafos

Foram implementadas duas estruturas de dados para representar grafos, sendo estas a Matriz de adjacência, que utiliza uma matriz bidimensional para armazenar o peso das arestas

e Lista de adjacência, utilizando um array de dicionários contendo apenas os vizinhos e seus respectivos pesos.

Nesse projeto o método de representação escolhido foi a lista de adjacência, porque redes de colaboração são tipicamente esparsas. Em um grafo com N usuários, cada um interage apenas com uma pequena fração dos demais, tornando a lista de adjacência uma alternativa mais eficiente em consumo de memória.

4. Análise dos resultados

A visualização base do grafo foi realizada utilizando a plataforma *open source* Gephi, voltada para a visualização e análise de grafos e redes. Muito utilizada na ciência de dados e ciência da computação, garante representações visuais confiáveis e precisas.

Os vários recursos da plataforma permitem que seja realizada uma análise exploratória inicial da rede, servindo como uma base para o desenvolvimento de métodos de análise próprios. O grafo construído a partir dos dados coletados, representado na Figura 2, foi construído com o *layout ForceAtlas2*, utilizando a modularidade para a coloração dos vértices e o *PageRank* para o tamanho no range de 5 a 50

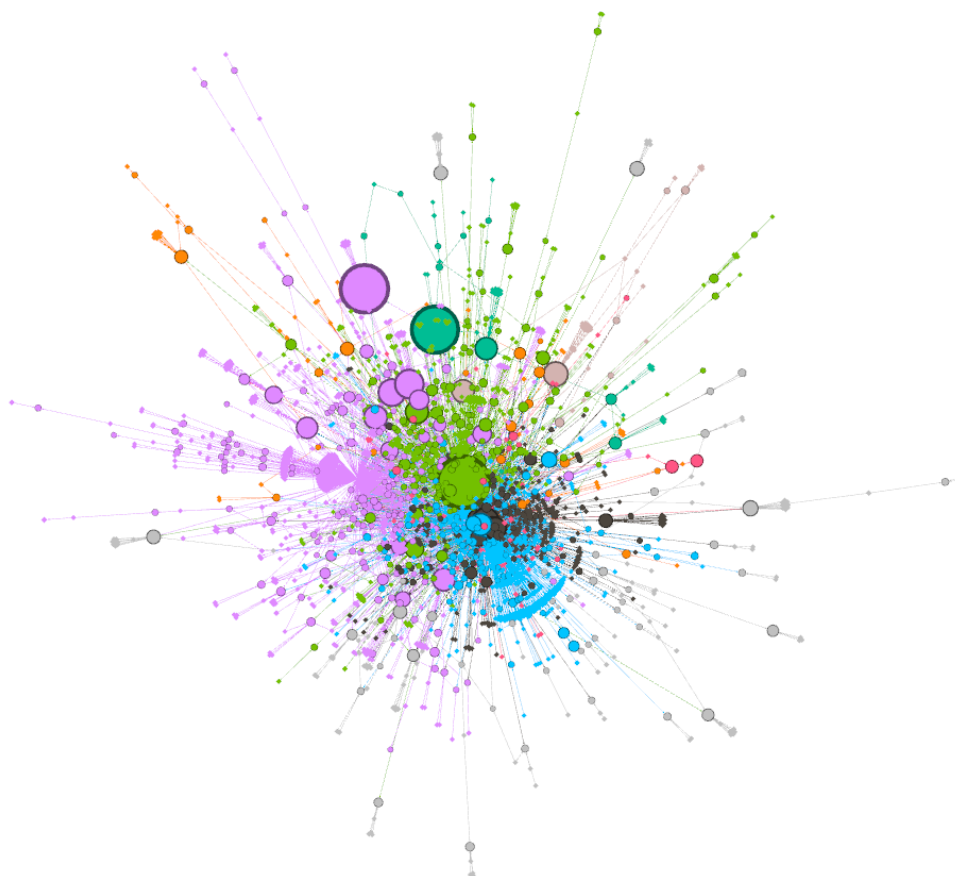


Figura 2. Visualização do grafo na plataforma Gephi

Também foram realizadas análises utilizando o grafo construído, levantando os seguintes dados:

- Top 10 usuários por grau de entrada

- Top 10 usuários por pagerank
- Top 10 Usuários por Centralidade de Proximidade
- Top 5 Usuários-Ponte
- 5 Maiores Comunidades Detectadas

4.1. Usuários por grau de entrada

Para determinar os usuários centrais e mais ativos na comunidade, foi implementado um método *degree centrality* que calculou o grau de entrada de cada um dos usuários. Essa métrica representa o número de interações que o usuário em questão recebeu de outros membros da comunidade, por exemplo comentários, *reviews* e *merges*.

O método percorre todos os vértices do grafo, conta quantas arestas apontam para cada uma delas. Essa métrica é capaz de identificar usuários que frequentemente recebem interações, comportamento tipicamente observado em mantenedores e contribuintes chave.

4.2. Usuários por page rank

Para caracterizar os usuários mais influentes no projeto foi utilizada a métrica *page rank*, através do método de mesmo nome que, calcula a relevância de cada contribuidor pela quantidade e qualidade de interações que o mesmo recebeu. Um usuário é considerado influente quando as interações que recebe são majoritariamente provenientes de outros usuários relevantes.

O método utiliza um algoritmo Power Iteration com *Damping factor* de 0.85, que executa repetidamente 100 interações até convergir. No momento inicial todos os usuários começam com o mesmo *Page Rank*. A cada interação o *Page Rank* é calculado somando as contribuições dos usuários que interagiram com ele.

4.3. Usuários por centralidade

A identificação dos usuários mais bem posicionados na rede foi realizada através da implementação do método *closeness centrality*, que calcula o quão próximo cada usuário está de todos os demais contribuidores. Aqueles com alta proximidade conseguem alcançar rapidamente outros através de caminhos curtos de interações.

O método é baseado em um algoritmo de busca em largura para calcular a menor distância de um usuário para todos os outros alcançáveis no grafo. Para cada um dos contribuidores é somada a distância e aplicadas as fórmula de Wasserman e Faust.

Os usuários detectados como centrais são considerados centralmente localizados, funcionando como pontos de convergência para a comunicação e colaboração. Usuários com alta proximidade podem disseminar informações com muito mais agilidade e eficiência que usuários normais.

4.4. Usuários ponte

Para caracterizar usuários que se com diferentes partes do projeto foi utilizado um método para calcular quantas interações cada um realiza com membros de comunidades diferentes da sua. O método utiliza o grafo completo e a atribuição de comunidade do usuário calculada pelo *Label Propagation*. Para cada contribuidor, identifica-se sua comunidade

e examina todos os seus sucessores (usuários com quem ele interage frequentemente). Ao realizar a contagem de quantos sucessores pertencem a comunidades diferentes é gerada uma pontuação de *bridging*.

Usuários identificados como pontes podem ser tech leads, desenvolvedores full stack ou coordenadores que trabalha de forma transversal em múltiplas áreas do projeto.

4.5. Maiores comunidades conectadas

A detecção de comunidades emergentes dentro do projeto foi realizada detectando grupos de usuários densamente conectados entre si, formando assim comunidades orgânicas dentro da rede de colaborações. Grupos de usuários como esses podem caracterizar sub-equipes, áreas funcionais do projeto ou grupos que trabalham juntos frequentemente.

O algoritmo utilizado é baseado no *Label Propagation*, que inicializa cada usuário com sua própria comunidade e, de forma iterativa, propaga os rótulos de comunidade pela rede. A cada iteração, os usuários são processados de forma aleatória e cada um adota o rótulo de comunidade mais comum entre seus vizinhos. O processo se repete continuamente até que não hajam mais mudanças de rótulo ou até que seja atingido o limite de 20 interações.

Essa métrica é essencial para entender a estrutura modular do projeto, identificando grupos de colaboradores que interagem intensamente entre si. Comunidades grandes costumam caracterizar equipes principais no projeto ou módulos inteiros enquanto comunidades menores costumam ser grupos específicos e especializados.

5. References

Bibliographic references must be unambiguous and uniform. We recommend giving the author names references in brackets, e.g. [Knuth 1984], [Boulic and Renault 1991], and [Smith and Jones 1999].

The references must be listed using 12 point font size, with 6 points of space before each reference. The first line of each reference should not be indented, while the subsequent should be indented by 0.5 cm.

Referências

Boulic, R. and Renault, O. (1991). 3d hierarchies for animation. In Magnenat-Thalmann, N. and Thalmann, D., editors, *New Trends in Animation and Visualization*. John Wiley & Sons Ltd.

Knuth, D. E. (1984). *The T_EX Book*. Addison-Wesley, 15th edition.

Smith, A. and Jones, B. (1999). On the complexity of computing. In Smith-Jones, A. B., editor, *Advances in Computer Science*, pages 555–566. Publishing Press.